

scRNA-Live Data Guide for Members

A guide to the single-cell data published as [TileDB-SOMA](#) objects in [s3://rancho-scrna-live/](#): what is there, how it is organized, and how to query it.

Inventory captured **2026-09-17**. Covers batches 1 through 13, processed with the **scAlliance1** pipeline.

1. What is in the bucket

Everything lives under [s3://rancho-scrna-live/](#), in three prefixes:

Prefix	What it holds
tiledb-experiments/	The data. One TileDB-SOMA Experiment per dataset, holding raw counts, normalized data, cell and gene annotation, and embeddings. Organized batch1/ through batch13/ .
collections/	Six TileDB-SOMA Collections , lightweight groups that <i>reference</i> a curated set of experiments, split by organism and platform.
manifests/	One Parquet manifest per collection: an aggregated index of every cell's obs metadata across that collection, for fast metadata queries without opening a single experiment.

How the pieces relate

```

tiledb-experiments/batch1/GSE123813/tiledbsoma_expt    <- the physical
Experiment (the data)
    ^
    | referenced by
collections/Human_10x/                                (a SOMA group: batch1_GSE123813, batch10_E-
MTAB-12339, ...)
    |
    | obs aggregated into
    v
manifests/Human_10x_manifest.parquet    (one row per cell, 117 obs columns +
4 source tags)

```

- **Experiments** are the source of truth and hold every expression matrix.
- **Collections** are groups of references, a few KB each. Opening one copies nothing.
- **Manifests** are a flat metadata cache. Filter there first, then open the experiments you actually need to pull expression from.

2. Inventory

Collection	Experiments	Cells	Manifest size
Human_10x	533	43,773,974	1.61 GB
Mouse_10x	202	12,467,871	429.3 MB
Human_ss2	23	66,641	2.7 MB
Mouse_ss2	15	12,335	0.6 MB
Rhesus_10x	4	258,710	7.7 MB
Cynomolgus_10x	3	345,562	12.7 MB
Total	780	56,925,093	—
Metric	Value		
Experiments published	1,026 (batches 1 through 13)		
Experiments referenced by a collection	780		
Total data size	2.26 TB		
S3 objects	744,771		
Columns per manifest	121 (117 obs + 4 source tags)		

Start with the small manifests. [Mouse_ss2](#) (0.6 MB) and [Human_ss2](#) (2.7 MB) load in seconds and share the identical 121-column schema, so they are the cheapest way to learn the layout. Select only the columns you need before touching [Human_10x](#).

Not every experiment is in a collection

1,026 experiments are published but only **780** are referenced by a collection. The other **246 are excluded on purpose**: they began from **author-provided count matrices** rather than being reprocessed from raw FASTQ through the scAlliance1 pipeline, so their gene sets and upstream QC are not consistent with the pipeline-processed datasets.

Those 246 experiments are still readable at their [tildeb-experiments/](#) URIs. They are simply not referenced by any collection, so they do not appear in the manifests or in any metric above. If you need them, address them directly by URI.

3. What is new in this release

Cell Ranger 7 reprocessing, batches 1 through 4. The 74 CellRanger-processed datasets in those batches were reprocessed, and counts now include intronic reads. Measured against the original Cell Ranger 6 delivery for the 73 datasets that have one:

Metric	Median change	Datasets improved
Median genes per cell	+26.7%	67 of 73
Median UMIs per cell	+12.2%	65 of 73

Metric	Median change	Datasets improved
Cells per dataset	+8.1%	63 of 73
Median mitochondrial percent	-12.2%	65 of 72

Total cells across those datasets rose from 7,350,600 to 8,000,416, and median barcode concordance with the original processing is 98.0%, so the gains reflect added sensitivity rather than a different cell population.

Metadata and ontology harmonization. Over 7.6M rows updated across 44 metadata attributes and 643 standardized terms. All 13 batches now share one data model, with remaining gaps filled as **NA** rather than omitted, so a query written against one collection runs unchanged against another.

Cell type annotation was regenerated with CellTypist and scArches for ecosystem compatibility.

4. Ontology versions

Ontology	Version	Where it appears	Populated
CL (Cell Ontology)	v2025-12-17	Celltypist_Cell_Type_Ontology_ID, scArches_Cell_Type_Ontology_ID, author_cell_type_cell_ontology_id	Yes
UBERON	v2025-12-04	sample_tissue_uberont_id	Yes
DOID	v2025-12-23	donor_disease_doid_id	Yes
NCIT	v2025-08-06	sample_tissue_uberont_id, donor_disease_doid_id (used where UBERON or DOID has no term)	Yes
EFO	v3.86.0	sample_library_preparation_EFO_id	Yes
MONDO	v2026-01-06	donor_disease_mondo_id	Column present, not yet populated
PATO	v2025-02-01	sex_ontology_term_id	Column present, not yet populated

Cell line identifiers use **Cellosaurus** accessions (CVCL_*) in **sample_cell_line_accession_cellosaurus**. Treatments appear as free-text names in **donor_treatment_mesh** and **sample_treatment_mesh**; the paired *_mesh_id columns are not yet populated.

The disease and tissue columns are deliberately mixed-vocabulary: `donor_disease_doid_id` carries a `D0ID:` term where one exists and an `NCIT:` term otherwise, and `sample_tissue_uberoid_id` behaves the same way with `UBERON:` and `NCIT:`. Match on the prefix rather than assuming a single vocabulary per column.

5. Getting set up

You need the `tildebsoma` conda environment and AWS credentials with read access to the bucket.

```
conda env create -f tildebsoma_environment.yml # first time only
conda activate tildebsoma
```

The environment includes `tildebsoma`, `pandas`, `pyarrow`, `s3fs`, and `scanpy`.

Quick check:

```
import pandas as pd

manifest = pd.read_parquet(
    "s3://rancho-scrna-live/manifests/Mouse_ss2_manifest.parquet"
)
print(manifest.shape) # (12335, 121)
```

6. Common tasks

Runnable examples for all of these are in `tildebsoma_examples_add_manifest.ipynb`.

Explore cell metadata across a whole collection

```
import pandas as pd

cols = ["obs_id", "dataset_id", "sample_tissue",
        "Celltypist_Cell_Annotation", "donor_disease"]
man = pd.read_parquet(
    "s3://rancho-scrna-live/manifests/Human_ss2_manifest.parquet",
    columns=cols
)
man["Celltypist_Cell_Annotation"].value_counts().head(20)
```

Selecting `columns=` matters: the full `Human_10x` manifest is 1.61 GB.

Open one experiment

```
import tiledbsoma as soma

uri = "s3://rancho-scrna-live/tiledb-
experiments/batch1/GSE123813/tiledbsoma_expt"
with soma.Experiment.open(uri) as expt:
    obs = expt.obs.read().concat().to_pandas()
    var = expt.ms["RNA"].var.read().concat().to_pandas()
    print(expt.ms["RNA"].X.keys())      # ['counts', 'data']
```

List what a collection contains

```
with soma.Collection.open("s3://rancho-scrna-live/collections/Human_10x")
as coll:
    print(len(coll.keys()))
    print(list(coll.keys())[:5])      # ['batch10_E-MTAB-12339', ...]
```

Member keys are `<batch>_<dataset_id>`, which maps directly onto the `tiledb-experiments/<batch>/<dataset_id>/` path.

Query cells by annotation and pull expression

```
query = expt.axis_query(
    "RNA",
    obs_query=soma.AxisQuery(value_filter="Celltypist_Cell_Annotation == 'T
cell'"),
)
adata = query.to_anndata(X_name="counts")
```

Use `X_name="counts"` for raw counts and `"data"` for the normalized layer.

Find which experiments express a gene

Genes live on the `var` axis, not in `obs`, so this needs a `var_query`:

```
query = expt.axis_query(
    "RNA", var_query=soma.AxisQuery(value_filter="var_id == 'CD8A'")
)
```

7. Key identifiers

- **Experiment URI:** `s3://rancho-scrna-live/tiledb-experiments/<batch>/<dataset_id>/tiledbsoma_expt`
- **Collection URI:** `s3://rancho-scrna-live/collections/<Organism_Platform>`

- **Manifest URI:** `s3://rancho-scRNA-live/manifests/<Organism_Platform>_manifest.parquet`
- **Collection member key:** `<batch>_<dataset_id>`

Gene identifiers (in `var`):

Column	Meaning
<code>var_id</code>	Gene symbol, e.g. <code>CD8A</code>
<code>gene_ids</code>	Ensembl ID, e.g. <code>ENSG00000243485</code>
<code>mt</code>	Boolean, mitochondrial gene
<code>genome</code>	Reference genome, e.g. <code>GRCh38</code>
<code>highly_variable</code>	Boolean, from HVG selection

Cell type annotation (in `obs`):

Column	Meaning
<code>Celltypist_Cell_Annotation</code>	CellTypist label, with <code>_Conf_Score</code>
<code>scArches_Cell_Annotation</code>	scArches label, with <code>_Uncert</code>
<code>author_cell_type</code>	Original author label, where provided
<code>*_Cell_Type_Ontology_ID / _Name</code>	The CL term for each of the above

Expression layers: `X["counts"]` is raw counts, `X["data"]` is normalized. **Embeddings:** `obsm` holds `X_pca`, `X_pca_harmony`, and `X_umap`.

QC columns (in `obs`): `nCount_RNA`, `nFeature_RNA`, `percent.mt`, `doublet_score`, `predicted_doublet`, `high_mt`, `high_ct`, `basic_qc`.

8. Syncing to your own bucket

We recommend mirroring this data into your organization's own S3 bucket, so it can be distributed as an internal resource and so you keep it regardless of the status of the scRNA-Live engagement.

Two steps. Copy the data, then run one command to repoint the collections:

```
# 1. copy (2.26 TB across 744,771 objects, so allow time)
aws s3 sync s3://rancho-scRNA-live/ s3://your-bucket/

# 2. repoint every collection at your bucket
python rewrite_collection_uris.py s3://your-bucket --apply
```

That is the whole procedure. Drop `--apply` for a dry run first.

Why step 2 is needed

The three prefixes behave differently under a sync:

Prefix	Survives a sync?	Why
<code>tiledb-experiments/</code>	Yes, unchanged	An experiment stores its internal members (<code>obs</code> , <code>ms</code> , <code>X</code>) as <i>relative</i> paths, so it resolves wherever it lands.
<code>manifests/</code>	Yes, unchanged	Manifests hold no URIs at all, only the <code>batch</code> , <code>curation_dataset_id</code> , <code>experiment</code> , and <code>collection_name</code> tags.
<code>collections/</code>	No, run step 2	A collection stores each member as an <i>absolute</i> URI, so a copied collection still points at <code>s3://rancho-scna-live</code> .

The experiments sit in a sibling prefix rather than inside the collection's own prefix. TileDB only records a relative member path when the member sits underneath the group, and a relative `../` path does not resolve, so absolute URIs are the only option. This is a property of the layout, not a fault in any one collection.

Skipping step 2 fails silently while you still have access to our bucket. Queries against your synced collections will appear to work, but they stream from `s3://rancho-scna-live`, so you get no egress saving, no isolation, and no protection if that access ends. It only surfaces as a hard error, `DoesNotExistError: Cannot open group`, later on.

What step 2 does

`rewrite_collection_uris.py` discovers every collection under `s3://your-bucket/collections/`, rewrites each member URI, and:

- verifies every new target exists before writing anything, so an incomplete sync is caught rather than silently baked in,
- leaves collection membership, naming, and collection-level metadata untouched,
- is safe to re-run, reporting `already correct` on a second pass,
- reports anything it could not match and exits non-zero, so it can be used in a pipeline.

Your counts after the rewrite should match the table in [Inventory](#).

Confirm you are reading your own copy

```
import tiledbsoma as soma

with soma.Collection.open("s3://your-bucket/collections/Human_ss2") as coll:
    for name, (uri, _) in list(coll.members().items())[:3]:
        print(name, uri)          # every URI should start with s3://your-bucket
```

Any URI still showing `rancho-scrna-live` was not rewritten and will keep reading from our bucket.

Other forms

```
python rewrite_collection_uris.py s3://your-bucket/collections/Human_10x --
apply    # one collection
python rewrite_collection_uris.py /mnt/scrna-live --apply
# a local mirror
```

9. Getting help

Questions on the data, the schema, or query patterns: contact Anne Cooley, Technical Lead.
Project and delivery questions: contact Nicole Leyland, Project Manager.